

EC200U&EG91xU Series

QuecOpen (SDK)

SPI Development Guide

LTE Standard Module Series

Version: 1.1

Date: 2023-11-06

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2022-01-24	Ryan YI	Creation of the document
1.0	2022-03-21	Ryan YI	First official release
1.1	2023-11-06	Sum LI	1. Updated the document name based on the unified QuecOpen naming. 2. Added applicable modules EG912U-GL and EG915U series. 3. Added a note about DMA polling mode (Chapter 3.3.1.2). 4. Updated the SPI clock frequency enumeration: <code>ql_spi_clk_e</code> (Chapter 3.3.1.3). 5. Updated the SPI error code enumeration: <code>ql_errcode_spi_e</code> (Chapter 3.3.1.4). 6. Updated CS pin enumeration: <code>ql_spi_cs_sel_e</code> (Chapter 3.3.2.7). 7. Updated the log capture tool description (Chapter 4.2).

Contents

About the Document.....	3
Contents.....	4
Table Index.....	6
1 Introduction	7
1.1. Applicable Modules	7
2 SPI Development Preparation	8
3 SPI API.....	9
3.1. Header File	9
3.2. API Overview.....	9
3.3. API Description.....	10
3.3.1. ql_spi_nor_init	10
3.3.1.1. ql_spi_port_e	11
3.3.1.2. ql_spi_transfer_mode_e	11
3.3.1.3. ql_spi_clk_e	12
3.3.1.4. ql_errcode_spi_e	13
3.3.2. ql_spi_init_ext	14
3.3.2.1. ql_spi_config_s.....	15
3.3.2.2. ql_spi_input_mode_e	16
3.3.2.3. ql_spi_cs_pol_e.....	16
3.3.2.4. ql_spi_cpol_pol_e.....	16
3.3.2.5. ql_spi_cpha_pol_e.....	17
3.3.2.6. ql_spi_input_sel_e.....	18
3.3.2.7. ql_spi_cs_sel_e	18
3.3.2.8. ql_spi_clk_delay_e	19
3.3.3. ql_spi_read_follow_write.....	19
3.3.4. ql_spi_write_read	20
3.3.5. ql_spi_read.....	21
3.3.6. ql_spi_write	21
3.3.7. ql_spi_release	22
3.3.8. ql_spi_cs_low	22
3.3.9. ql_spi_cs_high	23
3.3.10. ql_spi_cs_auto	23
3.3.11. ql_spi_set_irq	23
3.3.11.1. ql_spi_irq_s.....	24
3.3.11.2. ql_spi_threshold_e	25
3.3.11.3. ql_spi_callback	25
3.3.12. ql_spi_get_tx_fifo_free	26
3.3.13. ql_spi_request_sys_clk.....	26
3.3.14. ql_spi_release_sys_clk	26

4 Example 28

4.1. Development Example 28

4.2. Feature Debugging..... 28

5 Appendix References 30

Table Index

Table 1: Applicable Modules..... 7

Table 2: API Overview 9

Table 3: Related Documents..... 30

Table 4: Terms and Abbreviations 30

1 Introduction

Quectel EC200U and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS system. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on CSDK build environment. It introduces SPI development preparation, SPI API, development example and debugging of Quectel EC200U and EG91xU family modules in QuecOpen® solution.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 SPI Development Preparation

The following firmware library is required to implement SPI on EC200U and EG91xU family modules in QuecOpen solution.

- `#include "ql_api_spi.h"`

3 SPI API

3.1. Header File

ql_api_spi.h, the header file of SPI API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

3.2. API Overview

Table 2: API Overview

Function	Description
<i>ql_spi_init()</i>	Initializes SPI.
<i>ql_spi_init_ext()</i>	Initializes SPI (SPI bus parameters configuration).
<i>ql_spi_read_follow_write()</i>	Sets data to be sent and then received through SPI.
<i>ql_spi_write_read()</i>	Sets data to be sent and received simultaneously through SPI.
<i>ql_spi_read()</i>	Sets data to be received through SPI.
<i>ql_spi_write()</i>	Sets data to be sent through SPI.
<i>ql_spi_release()</i>	Releases SPI bus.
<i>ql_spi_cs_low()</i>	Forces to pull down the CS pin of SPI.
<i>ql_spi_cs_high()</i>	Forces to pull up the CS pin of SPI.
<i>ql_spi_cs_auto()</i>	Sets automatic control of CS pin through the SPI controller.
<i>ql_spi_set_irq()</i>	Sets SPI to use DMA interrupt callback function.
<i>ql_spi_get_tx_fifo_free()</i>	Gets the number of idle data in SPI TX FIFO.

<code>ql_spi_request_sys_clk()</code>	Requests not to enter slow clock.
<code>ql_spi_release_sys_clk()</code>	Releases the request not to enter slow clock.

3.3. API Description

3.3.1. ql_spi_nor_init

This function initializes SPI and must be called before other SPI API. Before calling this function, you need to set the specified GPIO pin to SPI by `ql_pin_set_func()`. See **document [2]** for details.

Calling `ql_spi_nor_init()` will configure `ql_spi_input_mode_e` to `QL_SPI_INPUT_TRUE`, `framesize` to 8, `ql_spi_cs_pol_e` to `QL_SPI_CS_ACTIVE_LOW`, `ql_spi_cpol_pol_e` to `QL_SPI_CPOL_LOW`, `ql_spi_cpha_pol_e` to `QL_SPI_CPHA_1Edge`, `ql_spi_input_sel_e` to `QL_SPI_DI_1`, `ql_spi_cs_sel_e` to `QL_SPI_CS0`, and `ql_spi_clk_delay_e` to `QL_SPI_CLK_DELAY_0` by default. See **Chapter 3.3.2** for details.

● Prototype

```
ql_errcode_spi_e ql_spi_init(ql_spi_port_e port, ql_spi_transfer_mode_e transmode, ql_spi_clk_e spiclk)
```

● Parameter

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

transmode:

[In] SPI transfer mode. See **Chapter 3.3.1.2** for details.

spiclk:

[In] SPI clock frequency. See **Chapter 3.3.1.3** for details.

● Return Value

See **Chapter 3.3.1.4** for details.

3.3.1.1. ql_spi_port_e

The enumeration of SPI bus:

```
typedef enum
{
    QL_SPI_PORT1,
    QL_SPI_PORT2,
}ql_spi_port_e
```

● Member

Member	Description
<i>QL_SPI_PORT1</i>	SPI1 bus
<i>QL_SPI_PORT2</i>	SPI2 bus

3.3.1.2. ql_spi_transfer_mode_e

The enumeration of SPI transfer modes:

```
typedef enum
{
    QL_SPI_DIRECT_POLLING = 0,
    QL_SPI_DIRECT_IRQ,
    QL_SPI_DMA_POLLING,
    QL_SPI_DMA_IRQ,
}ql_spi_transfer_mode_e
```

● Member

Member	Description
<i>QL_SPI_DIRECT_POLLING</i>	FIFO read/write. Polling waiting.
<i>QL_SPI_DIRECT_IRQ</i>	FIFO read/write. Interrupt notification. (Currently not supported.)
<i>QL_SPI_DMA_POLLING</i>	DMA read/write. Polling waiting.
<i>QL_SPI_DMA_IRQ</i>	DMA read/write. Interrupt notification.

NOTE

When the DMA polling mode is applied by EC200U or EG91xU family modules, the SPI will preempt the DMA channel with the SD card, therefore if the SD card has occupied the DMA channel, SPI initialization fails.

3.3.1.3. ql_spi_clk_e

The enumeration of SPI clock frequency:

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_97_656KHZ_MIN = 97656,
    QL_SPI_CLK_100KHZ = 100000,
    QL_SPI_CLK_781_25KHZ = 781250,
    QL_SPI_CLK_1_5625MHZ = 1562500,
    QL_SPI_CLK_3_125MHZ = 3125000,
    QL_SPI_CLK_5MHZ = 5000000,
    QL_SPI_CLK_6_25MHZ = 6250000,
    QL_SPI_CLK_10MHZ = 10000000,
    QL_SPI_CLK_12_5MHZ = 12500000,
    QL_SPI_CLK_20MHZ = 20000000,
    QL_SPI_CLK_25MHZ = 25000000,
    QL_SPI_CLK_33_33MHZ = 33000000,
    QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e
```

- **Member**

Member	Description
<i>QL_SPI_CLK_INVALID</i>	Invalid parameter.
<i>QL_SPI_CLK_97_656KHZ_MIN</i>	Clock frequency: 97.656 kHz
<i>QL_SPI_CLK_100KHZ</i>	Clock frequency: 100 kHz
<i>QL_SPI_CLK_781_25KHZ</i>	Clock frequency: 781.25 kHz
<i>QL_SPI_CLK_1_5625MHZ</i>	Clock frequency: 1.5625 MHz
<i>QL_SPI_CLK_3_125MHZ</i>	Clock frequency: 3.125 MHz
<i>QL_SPI_CLK_5MHZ</i>	Clock frequency: 5 MHz

QL_SPI_CLK_6_25MHZ	Clock frequency: 6.25 MHz
QL_SPI_CLK_10MHZ	Clock frequency: 10 MHz
QL_SPI_CLK_12_5MHZ	Clock frequency: 12.5 MHz
QL_SPI_CLK_20MHZ	Clock frequency: 20 MHz
QL_SPI_CLK_25MHZ	Clock frequency: 25 MHz
QL_SPI_CLK_33_33MHZ	Clock frequency: 33.33 MHz
QL_SPI_CLK_50MHZ_MAX	Clock frequency: 50 MHz (Maximum)

3.3.1.4. ql_errcode_spi_e

The enumeration of result codes of SPI:

```
typedef enum
{
    QL_SPI_SUCCESS                = 0,
    QL_SPI_ERROR                  = 1 | (QL_COMPONENT_BSP_SPI << 16),
    QL_SPI_PARAM_TYPE_ERROR,
    QL_SPI_PARAM_DATA_ERROR,
    QL_SPI_PARAM_ACQUIRE_ERROR,
    QL_SPI_PARAM_NULL_ERROR,
    QL_SPI_DEV_NOT_ACQUIRE_ERROR,
    QL_SPI_PARAM_LENGTH_ERROR,
    QL_SPI_MALLOC_MEM_ERROR,
    QL_SPI_ADDR_ALIGNED_ERROR,
    QL_SPI_MUTEX_CREATE_ERROR,
    QL_SPI_MUTEX_LOCK_ERROR,
    QL_SPI_SET_GPIO_ERROR,
    QL_SPI_DEV_BUSY_ERROR,
}ql_errcode_spi_e
```

● Member

Member	Description
QL_SPI_SUCCESS	Successful execution.
QL_SPI_ERROR	Failed execution.
QL_SPI_PARAM_TYPE_ERROR	Parameter type error.

<code>QL_SPI_PARAM_DATA_ERROR</code>	Parameter data error.
<code>QL_SPI_PARAM_ACQUIRE_ERROR</code>	Unable to get parameter.
<code>QL_SPI_PARAM_NULL_ERROR</code>	Parameter pointer is NULL.
<code>QL_SPI_DEV_NOT_ACQUIRE_ERROR</code>	Unable to get SPI bus.
<code>QL_SPI_PARAM_LENGTH_ERROR</code>	Parameter length error.
<code>QL_SPI_MALLOC_MEM_ERROR</code>	Memory allocation error.
<code>QL_SPI_ADDR_ALIGNED_ERROR</code>	Address not 4-byte aligned.
<code>QL_SPI_MUTEX_CREATE_ERROR</code>	Failed mutex creation.
<code>QL_SPI_MUTEX_LOCK_ERROR</code>	Mutex lock timeout.
<code>QL_SPI_SET_GPIO_ERROR</code>	Set GPIO error.
<code>QL_SPI_DEV_BUSY_ERROR</code>	Device busy.

3.3.2. ql_spi_init_ext

This function initializes SPI (SPI bus parameters configuration) and must be called before other SPI APIs.

This function selects SPI bus, configure SPI input, clock frequency, frame size, CS pin and pin level, clock polarity, clock phase, data input pin, transfer mode and MISO delay sampling. Before calling this function, you need to set relevant GPIO pins to SPI by `ql_pin_set_func()`. See **document [2]** for details.

- **Prototype**

```
ql_errcode_spi_e ql_spi_init_ext(ql_spi_config_s spi_config)
```

- **Parameter**

spi_config:

[In] SPI bus parameters configuration. See **Chapter 3.3.2.1** for details.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.2.1. ql_spi_config_s

The structure of SPI bus configuration parameters:

```
typedef struct
{
    ql_spi_input_mode_e input_mode;
    ql_spi_port_e port;
    unsigned int framesize;
    ql_spi_clk_e spiclk;
    ql_spi_cs_pol_e cs_polarity0;
    ql_spi_cs_pol_e cs_polarity1;
    ql_spi_cpol_pol_e cpol;
    ql_spi_cpha_pol_e cpha;
    ql_spi_input_sel_e input_sel;
    ql_spi_transfer_mode_e transmode;
    ql_spi_cs_sel_e cs;
    ql_spi_clk_delay_e clk_delay;
} ql_spi_config_s
```

● Parameter

Type	Parameter	Description
<i>ql_spi_input_mode_e</i>	<i>input_mode</i>	SPI input. See Chapter 3.3.2.2 for details.
<i>ql_spi_port_e</i>	<i>port</i>	SPI bus. See Chapter 3.3.1.1 for details.
unsigned int	<i>framesize</i>	Frame size. Range: 0–32. Default: 8. Unit: bit.
<i>ql_spi_clk_e</i>	<i>spiclk</i>	SPI clock frequency. See Chapter 3.3.1.3 for details.
<i>ql_spi_cs_pol_e</i>	<i>cs_polarity0</i>	CS0 pin level. See Chapter 3.3.2.3 for details.
<i>ql_spi_cs_pol_e</i>	<i>cs_polarity1</i>	CS1 pin level. See Chapter 3.3.2.3 for details.
<i>ql_spi_cpol_pol_e</i>	<i>cpol</i>	Clock polarity. See Chapter 3.3.2.4 for details.
<i>ql_spi_cpha_pol_e</i>	<i>cpha</i>	Clock phase. See Chapter 3.3.2.5 for details.
<i>ql_spi_input_sel_e</i>	<i>input_sel</i>	Data input pin. See Chapter 3.3.2.6 for details.
<i>ql_spi_transfer_mode_e</i>	<i>transmode</i>	SPI transfer mode. See Chapter 3.3.1.2 for details.
<i>ql_spi_cs_sel_e</i>	<i>cs</i>	CS pin. See Chapter 3.3.2.7 for details.
<i>ql_spi_clk_delay_e</i>	<i>clk_delay</i>	MISO delay sampling. See Chapter 3.3.2.8 for details.

3.3.2.2. ql_spi_input_mode_e

The enumeration of SPI input:

```
typedef enum
{
    QL_SPI_INPUT_FALSE,
    QL_SPI_INPUT_TRUE,
}ql_spi_input_mode_e
```

● Member

Member	Description
<i>QL_SPI_INPUT_FALSE</i>	Disable SPI input (read).
<i>QL_SPI_INPUT_TRUE</i>	Enable SPI input (read).

3.3.2.3. ql_spi_cs_pol_e

The enumeration of CS pin levels:

```
typedef enum
{
    QL_SPI_CS_ACTIVE_HIGH,
    QL_SPI_CS_ACTIVE_LOW,
}ql_spi_cs_pol_e
```

● Member

Member	Description
<i>QL_SPI_CS_ACTIVE_HIGH</i>	High level.
<i>QL_SPI_CS_ACTIVE_LOW</i>	Low level.

3.3.2.4. ql_spi_cpol_pol_e

The enumeration of clock polarities:

```
typedef enum
{
    QL_SPI_CPOL_LOW = 0,
```

```

    QL_SPI_CPOL_HIGH,
} ql_spi_cpol_pol_e

```

● Member

Member	Description
<i>QL_SPI_CPOL_LOW</i>	When SPI is not enabled, the CLK line is low level and the first edge is rising edge.
<i>QL_SPI_CPOL_HIGH</i>	When SPI is not enabled, the CLK line is high level and the first edge is falling edge.

3.3.2.5. ql_spi_cpha_pol_e

The enumeration of clock phases:

```

typedef enum
{
    QL_SPI_CPHA_1Edge,
    QL_SPI_CPHA_2Edge,
}ql_spi_cpha_pol_e

```

● Member

Member	Description
<i>QL_SPI_CPHA_1Edge</i>	Data sampling starts from the first clock edge.
<i>QL_SPI_CPHA_2Edge</i>	Data sampling starts from the second clock edge.

ql_spi_cpha_pol_e must be used together with *ql_spi_cpol_pol_e* to form four modes of SPI, as shown in following table.

SPI Mode	<i>ql_spi_cpol_pol_e</i>	<i>ql_spi_cpha_pol_e</i>
SPI mode0	<i>QL_SPI_CPOL_LOW</i>	<i>QL_SPI_CPHA_1Edge</i>
SPI mode1	<i>QL_SPI_CPOL_LOW</i>	<i>QL_SPI_CPHA_2Edge</i>
SPI mode2	<i>QL_SPI_CPOL_HIGH</i>	<i>QL_SPI_CPHA_1Edge</i>
SPI mode3	<i>QL_SPI_CPOL_HIGH</i>	<i>QL_SPI_CPHA_2Edge</i>

3.3.2.6. ql_spi_input_sel_e

The enumeration of data input pins:

```
typedef enum
{
    QL_SPI_DI_0 = 0,
    QL_SPI_DI_1,
    QL_SPI_DI_2,
}ql_spi_input_sel_e
```

● Member

Member	Description
<i>QL_SPI_DI_0</i>	DI0 as data input pin (currently not supported).
<i>QL_SPI_DI_1</i>	DI1 as data input pin.
<i>QL_SPI_DI_2</i>	DI2 as data input pin (currently not supported).

3.3.2.7. ql_spi_cs_sel_e

The enumeration of CS pins:

```
typedef enum
{
    QL_SPI_CS0 = 0,
    QL_SPI_CS1,
    QL_SPI_CS2,
    QL_SPI_CS3,
    QL_SPI_GPIO,
}ql_spi_cs_sel_e;
```

● Member

Member	Description
<i>QL_SPI_CS0</i>	CS0 as SPI CS pin.
<i>QL_SPI_CS1</i>	CS1 as SPI CS pin.
<i>QL_SPI_CS2</i>	CS2 as SPI CS pin (currently not supported).
<i>QL_SPI_CS3</i>	CS3 as SPI CS pin (currently not supported).

QL_SPI_GPIO

CS functionality is implemented through regular GPIOs. (The corresponding CS pin can no longer be configured into SPI CS functionality, and should be configured into GPIO).

3.3.2.8. ql_spi_clk_delay_e

The enumeration of MISO delay sampling:

```
typedef enum
{
    QL_SPI_CLK_DELAY_0 = 0,
    QL_SPI_CLK_DELAY_1,
}ql_spi_clk_delay_e;
```

● Member

Member	Description
QL_SPI_CLK_DELAY_0	MISO sampling without delay.
QL_SPI_CLK_DELAY_1	MISO sampling with one-clock-edge delay.

3.3.3. ql_spi_read_follow_write

This function sets the data to be sent and then received through SPI. The CS pin remains active during the function sends and receives data.

● Prototype

```
ql_errcode_spi_e ql_spi_read_follow_write(ql_spi_port_e port, unsigned char *outbuf, unsigned int outlen, unsigned char *inbuf, unsigned int inlen)
```

● Parameter

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

outbuf:

[In] Data to be sent.

outlen:

[In] Length of the data to be sent. Unit: byte.

inbuf:

[Out] Data to be received.

inlen:

[In] Length of the data to be received. Unit: byte.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.4. ql_spi_write_read

This function sets data to be sent and received simultaneously through SPI.

- **Prototype**

```
ql_errcode_spi_e ql_spi_write_read(ql_spi_port_e port, unsigned char *inbuf, unsigned char *outbuf, unsigned int len)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

inbuf:

[Out] Received data.

outbuf:

[In] Data to be sent.

len:

[In] Length of the data to be sent and received simultaneously. Unit: byte.

- **Return Value**

See **Chapter 3.3.1.4** for details.

NOTE

Before and after calling this function, you need to call *ql_spi_cs_low()*, *ql_spi_cs_high()*, or *ql_spi_cs_auto()* to control the CS pin.

3.3.5. ql_spi_read

This function sets data to be received through SPI.

- **Prototype**

```
ql_errcode_spi_e ql_spi_read(ql_spi_port_e port, unsigned char *buf, unsigned int len)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

buf:

[Out] Received data.

len:

[In] Length of the received data. Unit: byte.

- **Return Value**

See **Chapter 3.3.1.4** for details.

NOTE

Before and after calling this function, you need to call *ql_spi_cs_low()*, *ql_spi_cs_high()*, or *ql_spi_cs_auto()* to control the CS pin.

3.3.6. ql_spi_write

This function sets data to be sent through SPI.

- **Prototype**

```
ql_errcode_spi_e ql_spi_write(ql_spi_port_e port, unsigned char *buf, unsigned int len)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

buf:

[In] Data to be sent.

len:

[In] Length of the data to be sent. Unit: byte.

- **Return Value**

See **Chapter 3.3.1.4** for details.

NOTE

Before and after calling this function, you need to call *ql_spi_cs_low()*, *ql_spi_cs_high()*, or *ql_spi_cs_auto()* to control the CS pin.

3.3.7. ql_spi_release

This function releases SPI bus.

- **Prototype**

```
ql_errcode_spi_e ql_spi_release(ql_spi_port_e port)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.8. ql_spi_cs_low

This function forces to pull down the CS pin of SPI.

- **Prototype**

```
ql_errcode_spi_e ql_spi_cs_low(ql_spi_port_e port)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.9. ql_spi_cs_high

This function forces to pull up the CS pin of SPI.

- **Prototype**

```
ql_errcode_spi_e ql_spi_cs_high(ql_spi_port_e port)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.10. ql_spi_cs_auto

This function sets automatic control of CS pin through the SPI controller.

- **Prototype**

```
ql_errcode_spi_e ql_spi_cs_auto(ql_spi_port_e port)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.11. ql_spi_set_irq

This function sets SPI to use DMA interrupt callback function, which can only be used in *QL_SPI_DMA_IRQ* mode.

- **Prototype**

```
ql_errcode_spi_e ql_spi_set_irq(ql_spi_port_e port, ql_spi_irq_s mask, ql_spi_callback callfunc)
```


● Parameter

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

mask:

[In] Interrupt source mask. See **Chapter 3.3.11.1** for details.

callfunc:

[In] DMA interrupt callback function. See **Chapter 3.3.11.3** for details.

● Return Value

See **Chapter 3.3.1.4** for details.

3.3.11.1.ql_spi_irq_s

The structure of interrupt source masks:

```
typedef struct
{
    unsigned int rx_ovf : 1;
    unsigned int tx_th : 1;
    unsigned int tx_dma_done : 1;
    unsigned int rx_th : 1;
    unsigned int rx_dma_done : 1;
    ql_spi_threshold_e tx_threshold;
    ql_spi_threshold_e rx_threshold;
}ql_spi_irq_s;
```

● Parameter

Type	Parameter	Description
unsigned int	<i>rx_ovf</i>	RX overflow.
unsigned int	<i>tx_th</i>	Enable to TX FIFO interrupt.
unsigned int	<i>tx_dma_done</i>	Interrupt after DMA sending is complete.
unsigned int	<i>rx_th</i>	RX FIFO interrupt.
unsigned int	<i>rx_dma_done</i>	DMA receiving completes.
<i>ql_spi_threshold_e</i>	<i>tx_threshold</i>	TX FIFO interrupt trigger levels. See Chapter 3.3.11.2 for details.

<code>ql_spi_threshold_e</code>	<code>rx_threshold</code>	RX FIFO interrupt trigger levels. See Chapter 3.3.11.2 for details.
---------------------------------	---------------------------	--

3.3.11.2.ql_spi_threshold_e

The enumeration of FIFO interrupt trigger levels:

```
typedef enum
{
    QL_SPI_TRIGGER_1_DATA,
    QL_SPI_TRIGGER_4_DATA,
    QL_SPI_TRIGGER_8_DATA,
    QL_SPI_TRIGGER_12_DATA,
}ql_spi_threshold_e;
```

● Member

Member	Description
<code>QL_SPI_TRIGGER_1_DATA</code>	1 byte/halfword
<code>QL_SPI_TRIGGER_4_DATA</code>	4 bytes/halfword
<code>QL_SPI_TRIGGER_8_DATA</code>	8 bytes/halfword
<code>QL_SPI_TRIGGER_12_DATA</code>	12 bytes/halfword

3.3.11.3.ql_spi_callback

This function is DMA interrupt callback function.

● Prototype

```
typedef void (*ql_spi_callback)(ql_spi_irq_s cause)
```

● Parameter

cause:

[In] Interrupt source mask. See **Chapter 3.3.11.1** for details.

● Return Value

None

3.3.12. ql_spi_get_tx_fifo_free

This function gets the number of idle data in SPI TX FIFO.

- **Prototype**

```
ql_errcode_spi_e ql_spi_get_tx_fifo_free(ql_spi_port_e port, unsigned int *tx_free)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

tx_free:

[Out] The number of idle data in SPI TX FIFO.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.13. ql_spi_request_sys_clk

This function requests not to enter slow clock.

- **Prototype**

```
ql_errcode_spi_e ql_spi_request_sys_clk(ql_spi_port_e port)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.4** for details.

3.3.14. ql_spi_release_sys_clk

This function releases the request not to enter slow clock.

- **Prototype**

```
ql_errcode_spi_e ql_spi_release_sys_clk(ql_spi_port_e port)
```

- **Parameter**

port:

[In] SPI bus. See **Chapter 3.3.1.1** for details.

- **Return Value**

See **Chapter 3.3.1.4** for details.

4 Example

This chapter introduces how to use the above API for SPI development and simple debugging in application layer. In this example, SPI time sequence is analyzed by using an oscilloscope or logic analyzer without the need to connect SPI peripherals.

4.1. Development Example

components\ql-application\spi\spi_demo.c, the demo file for SPI API, is provided in Quectel EC200U and EG91xU family modules QuecOpen CSDK. It includes SPI initialization, reading and writing data through SPI. The entry function is *ql_spi_demo_init()*, as shown below:

```
QLOSStatus ql_spi_demo_init(void)
{
    QLOSStatus err = QL_OSI_SUCCESS;
    |
    err = ql_rtos_task_create(&spi_demo_task, SPI_DEMO_TASK_STACK_SIZE, SPI_DEMO_TASK_PRIO, "ql_spi_demo", ql_spi_demo_task_pthread, NULL, SPI_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI_DEMO_LOG("demo_task created failed");
        return err;
    }
    return err;
}
```

The above example is disabled by default in *ql_init_demo_thread()*. Uncomment *ql_spi_demo_init()* if you need to run it, as shown below:

```
*
: #ifdef QL_APP_FEATURE_SPI
:     //ql_spi_demo_init();
: #endif
*
```

4.2. Feature Debugging

SPI feature can be debugged by using Quectel LTE OPEN EVB, and the time sequence of SPI can be analyzed by using oscilloscope or logic analyzer.

SPI debugging steps:

1. Download the compiled firmware to the module.

2. Connect the LTE OPEN EVB to the PC through the USB port with a USB cable.
3. Capture the log with the log capture tool “cooltools”. For details about log capture, see **document [3]**. For acquisition of the tool, please contact Quectel Technical Support.
4. The USB AP Log Port mainly displays the system debugging information. After you capture the log, you will be able to check the SPI development related information.

`ql_spi_demo_init()` runs automatically after the module is booted. You can check whether SPI time sequence is correct through oscilloscope or logic analyzer. In the case of a failed execution, you can view the reason for the failure according to the corresponding log information.

5 Appendix References

Table 3: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_GPIO_Development_Guide
[3] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 4: Terms and Abbreviations

Abbreviation	Description
AP	Application Processor
API	Application Programming Interface
CLK	Clock
CPHA	Clock Phase
CPOL	Clock Polarity
CS	Chip Select
DI	Data Input
DMA	Direct Memory Access
DO	Data Output
EVB	Evaluation Board
FIFO	First Input First Output
IoT	Internet of Things

MISO	Master In Slave Out
MOSI	Master Out Slave In
PC	Personal Computer
RTOS	Real-Time Operating System
RX	Receive
SDK	Software Development Kit
SPI	Serial Peripheral Interface
TX	Transmit/Transmission
USB	Universal Serial Bus
